SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Artificial Intelligence and Data Science

ASSESSMENT TOOL 2 – Case Study

Course Code:	DSA03	**Course Title:**	Natural Language Processing
CO Assessed:	CO3 – Precision	**Assessment:**	Assessment Tool 2 – Case Study
Weightage:	30% of CIA		
CO5 Statement:	Apply N-gram models, smoothing techniques, part-of-speech tagging systems and to evaluate effective syntactic analysis. (BL-3)		
Student Name:	VIMALA K	**Reg. No.:**	192424276
Section / Batch:	B.Tech AI&DS	**Date:**	14/8/26

Code of Conduct

*I **VIMALA K 192424276** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: 

Instructions

- Read all three case studies carefully before attempting the questions.
- Provide appropriate justifications and interpretations for every computed result.
- Use NLP concepts, algorithms, and terminology wherever applicable.
- Diagrams, flowcharts, tables, or mathematical formulations may be used to support your answers.
- Duration: 30 minutes.

Section / Topic	Focus	Case Study
N-Gram Language Models and Smoothing Techniques	Bigram and Trigram probability computation, Maximum Likelihood Estimation (MLE), Backoff models, Deleted Interpolation, Entropy calculation, uncertainty analysis, real-time next-word prediction	Case Study 1 – Smart Mobile Keyboard Prediction System
Part-of-Speech (POS) Tagging and Word Classes	Penn Treebank tagset, contextual ambiguity resolution, rule-based tagging, stochastic tagging (HMM), emission and transition probabilities, word class identification, chatbot language understanding	Case Study 2 – AI-Powered Customer Support Chatbot
Transformation-Based Tagging (Brill Tagger)	POS tag correction, transformation rules, contextual tagging, linguistic validation, tag sequence refinement, ambiguity handling	Case Study 3 – News Analytics and POS Tag Correction System
Corpus Statistics and Probabilistic Analysis	Word frequency counting, corpus-based probability estimation, entropy-based uncertainty measurement, tagging confidence evaluation	Case Study 3 – News Analytics and POS Tag Correction System
Integrated Syntactic Analysis and NLP Applications	Application of language models, POS tagging, probabilistic methods, corpus analysis, ambiguity resolution, and decision-making in real-world NLP systems	Case Studies 1, 2, and 3

Annexure A – Case Study

CASE STUDY 1: E-Commerce Smart Search and Product Prediction System

An e-commerce company is developing an AI-powered search and autocomplete system that predicts the next word when customers enter product-related queries. The language model is trained on the corpus:

"machine learning improves business, machine learning enables automation, machine learning drives innovation."

The development team uses **Bigram and Trigram Language Models, Backoff Models, Deleted Interpolation, and Entropy Analysis** to improve search prediction and handle unseen word sequences.

The following statistics are obtained from the corpus:

- C(machine) = 3
- C(machine learning) = 3
- C(learning improves) = 1
- C(learning enables) = 1
- C(learning drives) = 1
- C(learning improves business) = 1

The interpolation weights are:

- λ_1 = 0.5 (Trigram)
- λ_2 = 0.3 (Bigram)
- λ_3 = 0.2 (Unigram)

The prediction probabilities after the word **"learning"** are:

- P(improves) = 0.33
- P(enables) = 0.33
- P(drives) = 0.33

Questions

1. Compute the Maximum Likelihood Estimate (MLE) of the Bigram Probability **P(learning | machine)**. Interpret how this probability can influence the next-word prediction of an e-commerce search system.
2. A customer enters the unseen sequence **"machine learning transforms"**. Apply the Backoff Model and estimate the probability of the word **"transforms"** using lower-order N-grams. Explain why backoff is useful when customers enter previously unseen search queries.
3. Using Deleted Interpolation, calculate the probability of the sequence **"machine learning improves"** using the given interpolation weights. Explain how combining trigram, bigram, and unigram probabilities improves robustness when training data is sparse.
4. Calculate the Entropy of the prediction distribution **P(improves) = 0.33, P(enables) = 0.33, P(drives) = 0.33**. Interpret the entropy value and explain what it indicates about uncertainty in the search prediction system.

5. Develop a Python program using **NLTK** for N-gram generation, **NumPy** for probability computations, and the **Math** library for entropy calculation. The program should construct Unigram, Bigram, and Trigram models from the given corpus, compute the MLE Bigram probability, estimate an unseen sequence using Backoff, calculate the Deleted Interpolation probability using the given weights, compute entropy of the prediction distribution, and display all intermediate calculations and the final next-word prediction in a structured format.

Coding:

```
import nltk

import numpy as np

import math

from collections import Counter

from nltk.util import ngrams



nltk.download("punkt")



corpus = "machine learning improves business, machine learning enables automation, machine learning drives innovation."



words = nltk.word_tokenize(corpus.lower())



unigram = Counter(words)

bigram = Counter(ngrams(words, 2))

trigram = Counter(ngrams(words, 3))



total_words = len(words)



print("=" * 60)
```

```
print("E-COMMERCE SMART SEARCH AND PRODUCT PREDICTION")
print("=" * 60)

print("\nCORPUS")
print(corpus)

print("\nUNIGRAM COUNTS")
for word, count in unigram.items():
    print(word, "=", count)

print("\nBIGRAM COUNTS")
for pair, count in bigram.items():
    print(pair, "=", count)

print("\nTRIGRAM COUNTS")
for triple, count in trigram.items():
    print(triple, "=", count)

p_learning_machine = (
    bigram[("machine", "learning")]
    / unigram["machine"]
)

print("\n1. MLE BIGRAM PROBABILITY")
print("C(machine) =", unigram["machine"])
print(
    "C(machine, learning) =",
```

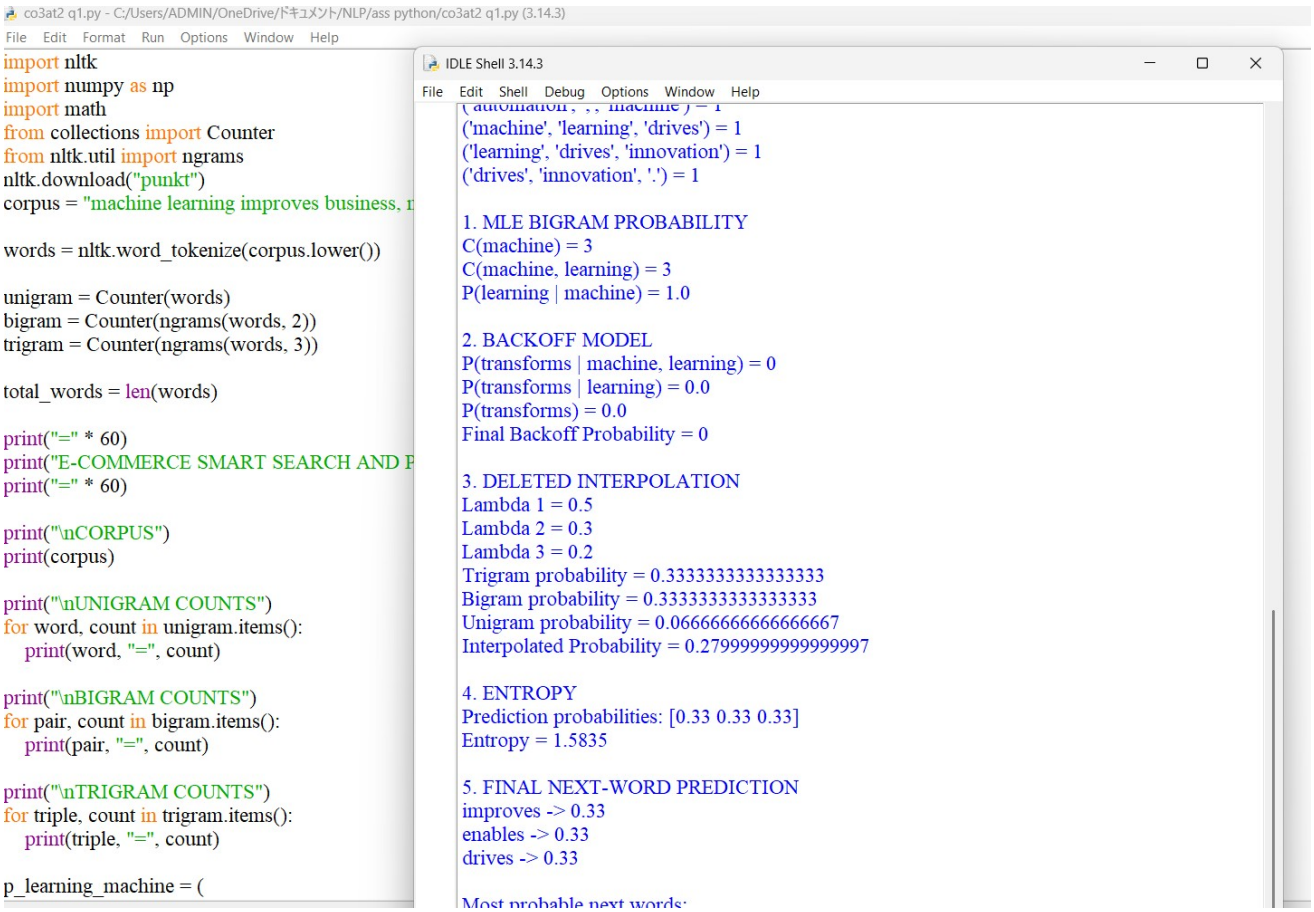
```
    bigram[("machine", "learning")]
)
print(
    "P(learning | machine) =",
    p_learning_machine
)

print("\n2. BACKOFF MODEL")

trigram_probability = 0

if ("machine", "learning", "transforms") in trigram:
    trigram_probability = (
        trigram[("machine", "learning", "transforms")]
        / bigram[("machine", "learning")]
    )
print(
    "P(transforms | machine, learning) =",
    trigram_probability
)

if unigram["learning"] > 0:
    bigram_probability = (
        bigram[("learning", "transforms")]
        / unigram["learning"]
    )
else:
```

```
    bigram_probability = 0

print(
    "P(transforms | learning) =",
    bigram_probability
)

unigram_probability = (
    unigram["transforms"] / total_words
)

print(
    "P(transforms) =",
    unigram_probability
)

backoff_probability = max(
    trigram_probability,
    bigram_probability,
    unigram_probability
)

print(
    "Final Backoff Probability =",
    backoff_probability
)
```

```
print("\n3. DELETED INTERPOLATION")

lambda1 = 0.5
lambda2 = 0.3
lambda3 = 0.2

p_tri = 1 / 3
p_bi = 1 / 3
p_uni = 1 / total_words

print("Lambda 1 =", lambda1)
print("Lambda 2 =", lambda2)
print("Lambda 3 =", lambda3)

print("Trigram probability =", p_tri)
print("Bigram probability =", p_bi)
print("Unigram probability =", p_uni)

interpolation_probability = (
    lambda1 * p_tri
    + lambda2 * p_bi
    + lambda3 * p_uni
)

print(
    "Interpolated Probability =",
    interpolation_probability
```

```
)

print("\n4. ENTROPY")

prediction_probabilities = np.array(
    [0.33, 0.33, 0.33]
)

entropy = 0

for probability in prediction_probabilities:
    entropy -= probability * math.log2(probability)

print(
    "Prediction probabilities:",
    prediction_probabilities
)

print(
    "Entropy =",
    round(entropy, 4)
)

print("\n5. FINAL NEXT-WORD PREDICTION")

predictions = {
```

```
  "improves": 0.33,
    "enables": 0.33,
    "drives": 0.33)sorted_predictions = sorted(
    predictions.items(),
    key=lambda x: x[1],
    reverse=True
)for word, probability in sorted_predictions:
    print(
        word,
        "->",
        probability
    )print("\nMost probable next words:")
print("improves, enables, drives")
print("\n" + "=" * 60)
```



CASE STUDY 2: AI-Powered Hospital Appointment Chatbot

A healthcare organization develops an AI-powered chatbot that assists patients in booking appointments. The chatbot performs **Part-of-Speech tagging** before identifying user intentions and generating responses.

Consider the following user inputs:

1. **"Book an appointment with the doctor."**
2. **"The book contains medical information."**

The chatbot uses the **Penn Treebank POS Tagset** and an **HMM-based probabilistic POS tagger** to determine the grammatical role of ambiguous words.

For the word **"book"**, the system is given:

- P(book | VB) = 0.7
- P(book | NN) = 0.3
- P(Start → VB) = 0.6
- P(Start → NN) = 0.4

The system must determine whether **"book"** functions as a Verb (VB) or Noun (NN) depending on its context.

Questions

1. Assign appropriate **Penn Treebank POS tags** to every word in both sentences. Justify the POS tag assigned to **"book"** using the grammatical context of each sentence.
2. Using the given HMM probabilities, compute the likelihood of **"book"** being tagged as a Verb (VB) and as a Noun (NN). Determine the most probable tag and explain why the probabilistic model favors that tag at the beginning of a sentence.
3. Compare **Rule-Based POS Tagging** and **Stochastic HMM Tagging** for resolving the ambiguity of the word **"book"**. Discuss the advantages and limitations of both approaches in a healthcare chatbot.
4. Evaluate how standardized POS tagsets such as the **Penn Treebank Tagset** can support downstream NLP tasks such as **intent detection, entity identification, and response generation** in healthcare chatbots.
5. Develop a Python program using **NLTK** to tokenize the given sentences and assign Penn Treebank POS tags. Implement a simple **HMM-based POS tagging calculation** using the given emission and transition probabilities. The program should calculate the probability of **"book"** being tagged as VB and NN, determine the most likely tag, display the tokenized sentences, POS-tagged output, intermediate probability calculations, and final tagging results in a structured format.

Coding:

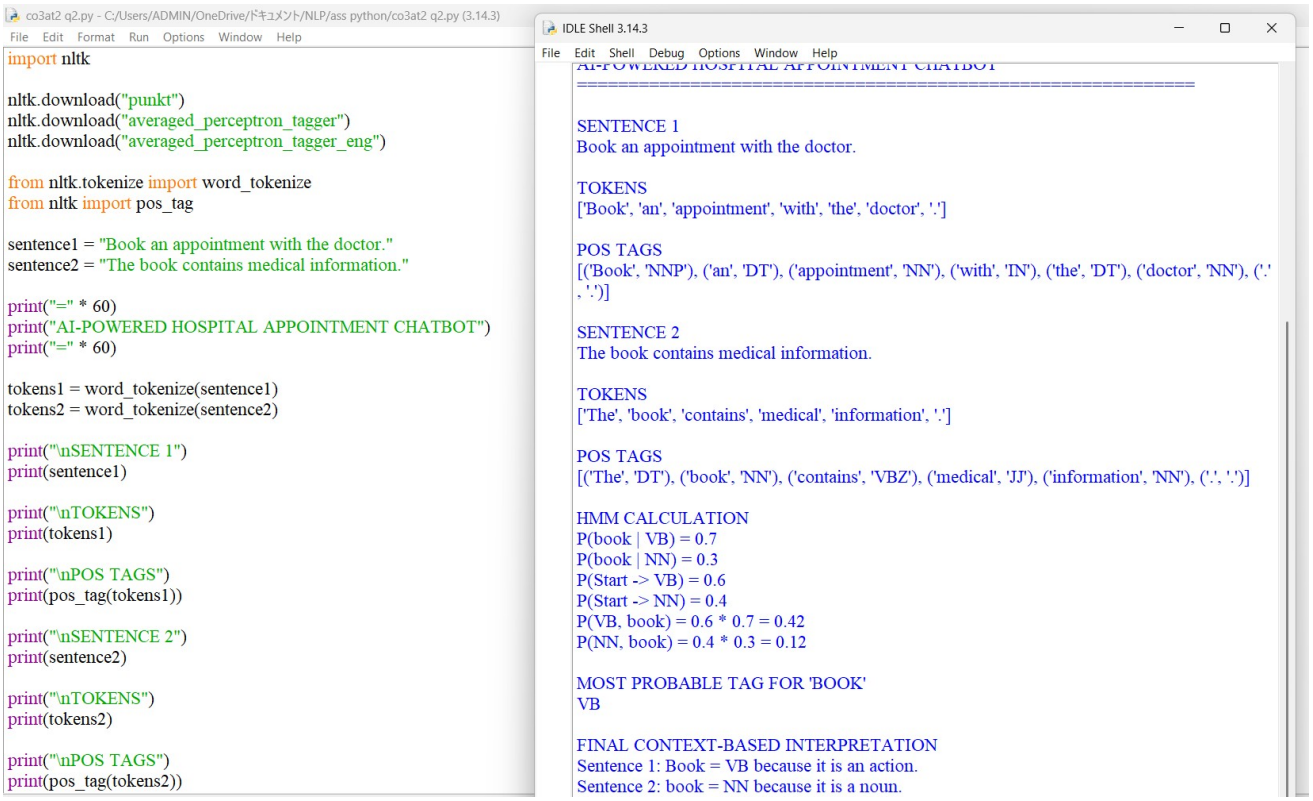
```
import nltk
```

```
nltk.download("punkt")

nltk.download("averaged_perceptron_tagger")

nltk.download("averaged_perceptron_tagger_eng")


from nltk.tokenize import word_tokenize

from nltk import pos_tag


sentence1 = "Book an appointment with the doctor."

sentence2 = "The book contains medical information."


print("=" * 60)

print("AI-POWERED HOSPITAL APPOINTMENT CHATBOT")

print("=" * 60)


tokens1 = word_tokenize(sentence1)

tokens2 = word_tokenize(sentence2)


print("\nSENTENCE 1")

print(sentence1)


print("\nTOKENS")

print(tokens1)


print("\nPOS TAGS")

print(pos_tag(tokens1))


print("\nSENTENCE 2")
```

```
print(sentence2)

print("\nTOKENS")
print(tokens2)

print("\nPOS TAGS")
print(pos_tag(tokens2))

p_book_vb = 0.7
p_book_nn = 0.3

p_start_vb = 0.6
p_start_nn = 0.4

prob_vb = p_start_vb * p_book_vb
prob_nn = p_start_nn * p_book_nn

print("\nHMM CALCULATION")

print(
    "P(book | VB) =",
    p_book_vb
)

print(
    "P(book | NN) =",
    p_book_nn
```

```
)


print(
    "P(Start -> VB) =",
    p_start_vb
)print(
    "P(Start -> NN) =",
    p_start_nn
)print(
    "P(VB, book) =",
    p_start_vb,
    "*",
    p_book_vb,
    "=",
    prob_vb
)print(
    "P(NN, book) =",
    p_start_nn,
    "*",
    p_book_nn,
    "=",
    prob_nn
)if prob_vb > prob_nn:
    best_tag = "VB"
else:
    best_tag = "NN"
print("\nMOST PROBABLE TAG FOR 'BOOK'")
```

```
print(best_tag)

print("\nFINAL CONTEXT-BASED INTERPRETATION")

print(

    "Sentence 1: Book = VB because it is an action."

)

print(

    "Sentence 2: book = NN because it is a noun."

)

print("\n" + "=" * 60)
```



CASE STUDY 3: Financial News POS Tag Correction and Uncertainty Analysis

A financial news analytics company processes thousands of financial news articles every day. The system initially assigns POS tags using a statistical POS tagger and subsequently applies a **Transformation-Based Tagger (Brill Tagger)** to correct common tagging errors.

The sentence processed by the system is:

"Market growth drives investment."

The initial POS tags produced by the system are:

- market/NN
- growth/NN
- drives/NNS
- investment/NN

A learned transformation rule states:

Change NNS to VBZ if the preceding word is tagged as NN.

The system also performs corpus-based frequency analysis and entropy-based uncertainty evaluation.

The corpus contains the following word frequencies:

1. market – 500 occurrences
2. growth – 350 occurrences
3. drives – 180 occurrences
4. investment – 420 occurrences

Questions

1. Apply the given transformation rule to the initial POS-tag sequence. Display the corrected POS-tag sequence and explain why the transformation is linguistically appropriate.
2. Analyze the initial tagging error for **"drives"**. Explain why the word should receive the **VBZ** tag in this sentence and discuss how the surrounding grammatical context helps identify the correct tag.
3. Using the given corpus statistics, calculate the **relative frequency/probability** of each word. Explain how corpus frequency information can support probabilistic POS tagging and language processing.
4. Calculate the **entropy of the word-frequency distribution** before and after applying the transformation rule. Explain what entropy indicates about uncertainty in the tagging or prediction process.
5. Develop a Python program using **NLTK** for initial POS tagging and implement a simple **Transformation-Based Tagger** that applies the rule **"Change NNS to VBZ if the preceding word is tagged as NN."** The program should tokenize the sentence, generate initial POS tags, apply the transformation rule, calculate word-frequency probabilities from the given corpus statistics, calculate entropy using the Python **math** library, and display the initial tags, corrected tags, frequency distribution, entropy values, and final POS-tagging results in a structured format.

Coding:

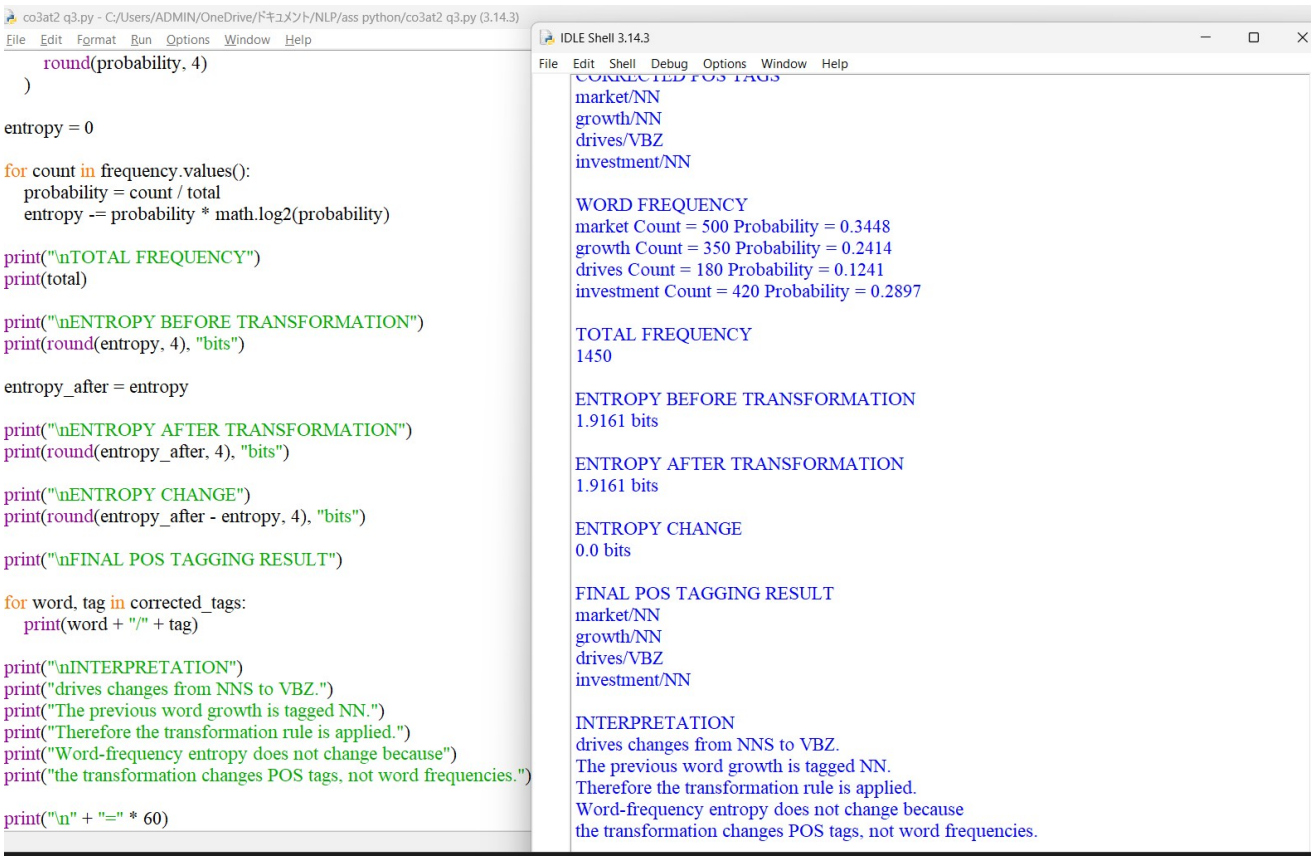
```
import nltk

import math



nltk.download("punkt")

nltk.download("averaged_perceptron_tagger")
```

```
nltk.download("averaged_perceptron_tagger_eng")

sentence = "Market growth drives investment."

words = nltk.word_tokenize(sentence)

initial_tags = [
    ("market", "NN"),
    ("growth", "NN"),
    ("drives", "NNS"),
    ("investment", "NN")
]

print("=" * 60)
print("FINANCIAL NEWS POS TAG CORRECTION")
print("=" * 60)

print("\nSENTENCE")
print(sentence)

print("\nTOKENS")
print(words)

print("\nINITIAL POS TAGS")

for word, tag in initial_tags:
    print(word + "/" + tag)
```

```
corrected_tags = initial_tags.copy()

for i in range(1, len(corrected_tags)):
    current_word = corrected_tags[i][0]
    current_tag = corrected_tags[i][1]
    previous_tag = corrected_tags[i - 1][1]

    if current_tag == "NNS" and previous_tag == "NN":
        corrected_tags[i] = (current_word, "VBZ")

print("\nTRANSFORMATION RULE")
print("Change NNS to VBZ if preceding word is NN.")

print("\nCORRECTED POS TAGS")

for word, tag in corrected_tags:
    print(word + "/" + tag)

frequency = {
    "market": 500,
    "growth": 350,
    "drives": 180,
    "investment": 420
}

total = sum(frequency.values())
```

```
print("\nWORD FREQUENCY")

for word, count in frequency.items():
    probability = count / total

    print(
        word,
        "Count =",
        count,
        "Probability =",
        round(probability, 4)
    )

entropy = 0

for count in frequency.values():
    probability = count / total
    entropy -= probability * math.log2(probability)

print("\nTOTAL FREQUENCY")
print(total)

print("\nENTROPY BEFORE TRANSFORMATION")
print(round(entropy, 4), "bits")

entropy_after = entropy
```

```
print("\nENTROPY AFTER TRANSFORMATION")

print(round(entropy_after, 4), "bits")


print("\nENTROPY CHANGE")

print(round(entropy_after - entropy, 4), "bits")


print("\nFINAL POS TAGGING RESULT")


for word, tag in corrected_tags:

    print(word + "/" + tag)


print("\nINTERPRETATION")

print("drives changes from NNS to VBZ.")

print("The previous word growth is tagged NN.")

print("Therefore the transformation rule is applied.")

print("Word-frequency entropy does not change because")

print("the transformation changes POS tags, not word frequencies.")


print("\n" + "=" * 60)
```


co3at2 q3.py - C:/Users/ADMIN/OneDrive/ドキュメント/NLP/ass python/co3at2 q3.py (3.14.3)
File Edit Format Run Options Window Help
round(probability, 4)
)
entropy = 0
for count in frequency.values():
probability = count / total
entropy -= probability * math.log2(probability)
print("\nTOTAL FREQUENCY")
print(total)
print("\nENTROPY BEFORE TRANSFORMATION")
print(round(entropy, 4), "bits")
entropy_after = entropy
print("\nENTROPY AFTER TRANSFORMATION")
print(round(entropy_after, 4), "bits")
print("\nENTROPY CHANGE")
print(round(entropy_after - entropy, 4), "bits")
print("\nFINAL POS TAGGING RESULT")
for word, tag in corrected_tags:
print(word + "/" + tag)
print("\nINTERPRETATION")
print("drives changes from NNS to VBZ.")
print("The previous word growth is tagged NN.")
print("Therefore the transformation rule is applied.")
print("Word-frequency entropy does not change because")
print("the transformation changes POS tags, not word frequencies.")
print("\n" + "=" * 60)
IDLE Shell 3.14.3
File Edit Shell Debug Options Window Help
market/NN
growth/NN
drives/VBZ
investment/NN
WORD FREQUENCY
market Count = 500 Probability = 0.3448
growth Count = 350 Probability = 0.2414
drives Count = 180 Probability = 0.1241
investment Count = 420 Probability = 0.2897
TOTAL FREQUENCY
1450
ENTROPY BEFORE TRANSFORMATION
1.9161 bits
ENTROPY AFTER TRANSFORMATION
1.9161 bits
ENTROPY CHANGE
0.0 bits
FINAL POS TAGGING RESULT
market/NN
growth/NN
drives/VBZ
investment/NN
INTERPRETATION
drives changes from NNS to VBZ.
The previous word growth is tagged NN.
Therefore the transformation rule is applied.
Word-frequency entropy does not change because
the transformation changes POS tags, not word frequencies.

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q. No. / Criteria Level	Understanding of Case	Application of Concepts	Analysis	Solution Design	Clarity	Sub. Total	Total %
1	7	6	7	6	6	32	93
2	6	7	6	7	6	32	
3	6	6	6	6	5	29	

Faculty In-charge	Course Coordinator	Program Director
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____